To use Laravel Octane with Swoole, we need to install this package: composer require laravel/Octane

To install the PHP Swoole extension: sudo apt install php8.4-openswoole

Then, we need to activate the extension in the php.ini file.

==========

To start the Octane server: php artisan Octane:start --host 0.0.0.0

==========

To define a repeated task in the app service provider:

```
Octane::tick('caching-query', function () {
    Log::info('caching-query.', ['timestamp' => now()]);
    $time = hrtime(true);

    $count = Event::count();
    $eventsInfo = Event::ofType('INFO')->get();
    $eventsWarning = Event::ofType('WARNING')->get();
    $eventsAlert = Event::ofType('ALERT')->get();

    $time = (hrtime(true) - $time) / 1_000_000;
    $result = [
        'count' => $count,
        'eventsInfo'=> $eventsInfo,
        'eventsWarning' => $eventsWarning,
        'eventsAlert'=> $eventsAlert,
        'cached_time' => $time,
    ];
    Cache::store('Octane')->put('cached-result-tick', $result);
})->seconds(60)->immediate();
```

==========

Note (From me): if the user defines the **Octane concurrent** function inside the **Octane tick**, an error will appear.

==========

To use the Octane concurrent function with the cache in Laravel:

```
Route::get('/httpcall/parallel', function () {
    $time = hrtime(true);
    [$sentenceJson, $nameJson] =Cache::store('Octane')->remember(
        'key-checking',20, function () {
            return Octane::concurrently([
                fn() => Http::get('http://127.0.0.1:8000/api/sentence')->json(),
                fn() => Http::get('http://127.0.0.1:8000/api/name')->json(),
            ]);
        }
    );
    $time = hrtime(true) - $time;

    return response()->json(
        array_merge(
            $sentenceJson,
            $nameJson,
            ['time_ms' => $time / 1_000_000]
        )
    );
});
```

==========

To create Octane table for caching:

```
Route::get('/table-create', function () {
    // Getting the table instance
    $table = Octane::table('my-table');
    // looping 1..90 creating rows with fake() helper
    for ($i = 1; $i <= 90; $i++) {
        $table->set(
            $i,
            [
                'uuid' => fake()->uuid(),
                'name' => fake()->name(),
                'age' => fake()->numberBetween(18, 99),
                'value' => fake()->randomFloat(2, 0, 1000)
            ]
        );
    }
    return "Table created!";
});
```

And in the Octane config file:

```
    'tables' => [
        'example:1000' => [
            'name' => 'string:1000',
            'votes' => 'int',
        ],
        'my-table:100' => [
            'uuid' => 'string:36',
            'name' => 'string:1000',
            'age' => 'int',
            'value' => 'float',
        ],
        'cache' => [
            'key' => 'string:256',
            'value' => 'string:2048576', // Increase this (e.g., 1MB)
            'expiration' => 'int:8',
        ],
        'request_context' => [
            'key' => 'string:256',
            'value' => 'string:2048576', // Increase this too
        ],
    ],
```

==========

```
Route::get('/table-get/', function () {
    $table = Octane::table('my-table');
    $row = $table->get(1);
    return $row;
});
```

```
Route::get('/table-get-all', function () {
    $table = Octane::table('my-table');
    $rows = [];
    foreach ($table as $key => $value) {
        $rows[$key] = $table->get($key);
    }
    //Add the table row count as the first row
    $rows[0] = count($table);
    return $rows;
});
```

```
Route::get('/metrics', function () {
    $server = App::make(Swoole\Http\Server::class);
    return $server->stats();
});
```

==========

Reloading the workers

Every time you make some changes to your application logic, you have to restart the application with the following command:

```
php artisan octane:reload
```

The `octane:reload` command restarts the workers instanced by Octane, and in this way, the code is reloaded.

==========

If you want to know about the status (running or not) of the Laravel Octane service, you can use the `octane:status` command:

```
php artisan octane:status
```

`octane:status` shows the **Octane server is running.** message if everything is fine. Otherwise, it shows **Octane server is not running.**:

==========

Listening events

Laravel Octane internally defines some events for notifying and managing when something happens, such as the worker starting, a request being received, and so on. In a framework, if you can produce some events, as a developer, you need a mechanism to listen for when an event occurs. The Laravel framework provides the developer with a mechanism to define and create events and a mechanism to listen (a listener). The listener in Laravel can execute a function defined by the developer.

==========

Looking into the Laravel Octane config file (config/octane.php) in the listener section, you will see that the defined events are as follows: WorkerStarting, RequestReceived, RequestHandled, RequestTerminated, TaskReceived, TaskTerminated, TickReceived, TickTerminated, OperationTerminated, WorkerErrorOccurred, and WorkerStopping.

==========

If you only want to install the packages from the `require` list (production), you can use the `--no-dev` option while you are installing the packages for the production environment:

```
composer install --optimize-autoloader --no-dev
```

Other than the `--no-dev` option, we are using `optimize-autoloader`. The option generates a data structure (an index) to map the class name, with the name of the file to be loaded.

The `optimize-autoloader` option reduces the bootstrap time.

==========

You should also reduce the number of log messages. Log messages are very useful for debugging purposes, but tracking the logs could be an expensive operation and could slow down the application.

For example, in the `.env` configuration file, typically, you can find the following:

```
LOG_LEVEL=debug
```

==========

The levels of debugging, from most detailed (with more logs) to just tracking the errors, are as follows: `debug`, `info`, `notice`, `warning`, `error`, `critical`, `alert`, and `emergency`. My suggestion is to change the level of logs (`LOG_LEVEL`) in a production environment to `error` in order to only track critical information because reducing the number of log messages makes your application faster and can reduce the amount of storage needed.

The general suggestion is to have a specific configuration for the production environment, so the advice is to create a file, `.env.prod`, where you can store your production configuration. Then, copy the `.env.prod` file on each deployment to your `.env` file in the production environment.

==========